

Conditional Trajectory Peaks: Single-Pass Multimodal Policies over Action Chunks

Anonymous Authors

Abstract—Multimodal imitation learning requires diverse executable futures under the same observation and consistent behavior across replanning cycles. We present Conditional Trajectory Peaks (CTP), a single-pass policy framework that jointly predicts complete action-chunk candidates, probability masses, and trajectory scales. Distribution-Aware Peak Specialization (DAPS) specializes trajectory peaks using trajectory-level posterior responsibilities and mass- and scale-modulated overlap constraints. Evidence-Gated Trajectory Belief Transport (ETBT) maintains cross-chunk consistency through geometric correspondence between exchangeable candidate sets, while allowing current policy evidence to override historical constraints. CTP achieves a coverage score of 91.40% on Push-T; success rates of 100.0%, 79.72%, and 84.44% on D3IL Avoiding, Aligning, and Sorting-2, respectively. On LIBERO, CTP achieves an average success rate of 97.25%. In real-world dual-arm experiments, CTP preserves both placement modes in a two-plate task, succeeding in all 50 trials. On bottle uprighting and pen placement into a holder, it maintains success rates comparable to $\pi_{0.5}$ while reducing policy inference latency from 218.24 ms to 75.80 ms. These results demonstrate that single-pass trajectory modeling can combine multimodal behavior, closed-loop consistency, and efficient inference.

Keywords—Multimodal imitation learning; structured trajectory modeling; action chunking; single-pass inference; mode preservation

I. INTRODUCTION

Robotic manipulation demonstrations are often multimodal: similar observations can admit different obstacle-avoidance paths, contact locations, pushing sequences, or subtask orders. Policies must represent these distinct executable futures rather than a single deterministic action. Squared-error behavioral cloning instead favors the conditional mean, which can enter low-probability or infeasible regions when trajectory modes are separated [1]. Diffusion Policy, BESO, and flow-matching policies model complex multimodal action distributions [2]–[5]. However, diffusion denoising and numerical flow integration generally require multiple network evaluations per replanning cycle. This iterative generation increases latency and computation in high-frequency closed-loop control. This raises a central question: *Can a policy generate trajectory-level multimodal actions with one network forward pass per replanning step?*

A direct alternative is to predict multiple trajectories in parallel. Existing multi-hypothesis policies demonstrate single-pass batched candidate generation [6], [7], but parallel outputs do not guarantee effective multimodality. Permutation symmetry can drive candidates toward the same dominant high-density region early in training. This produces *mode collapse*: nominally distinct candidates represent identical or similar behaviors, leaving the model effectively

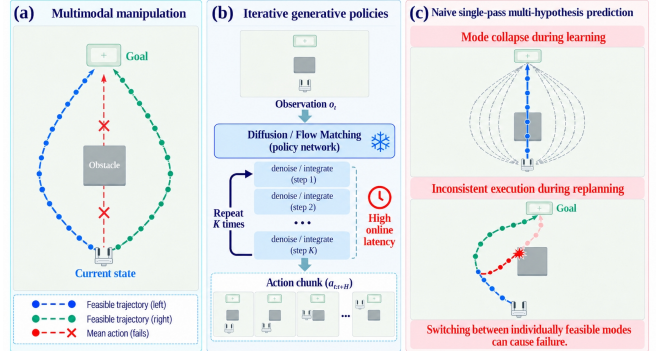


Fig. 1. Core challenges in multimodal closed-loop manipulation. (a) The same observation admits multiple feasible futures, while the conditional mean may be infeasible. (b) Diffusion and flow matching policies generate action chunks iteratively, increasing online decision latency. (c) Naive single-pass multi-candidate prediction can still suffer from mode collapse during training and inconsistent execution caused by mode switching during receding-horizon replanning.

unimodal. Additional output capacity therefore permits multiple modes without ensuring allocation to distinct data-supported regions. Fixed action anchors, offline clustering, or predefined mode quotas impose candidate roles through data processing or model design. Single-pass policies instead need data-driven specialization that assigns high-probability trajectories to distinct valid modes and reduces probability mass on unsupported redundant candidates.

Candidate diversity alone is insufficient for multimodal closed-loop control. Action-chunking policies execute only a chunk’s first few steps before replanning, so candidates must preserve behavioral continuity across decisions. Local probability changes can switch the policy between incompatible routes, stitching individually feasible chunks into discontinuous or unsuccessful behavior, a failure we call *cross-chunk mode stitching*. Unlike training-time mode collapse, stitching concerns behavioral identity across replanning cycles and can occur despite diverse candidates. Because finite mixture components are permutation-symmetric, their indices cannot identify corresponding behaviors across successive exchangeable candidate sets. We therefore formulate cross-chunk consistency as *behavioral identity correspondence* and *sequential transfer*: physical trajectory relationships identify continuations, while current policy evidence can revise or overturn earlier choices. Fig. 1 summarizes these challenges.

We therefore propose **Conditional Trajectory Peaks (CTP)**, a single-pass policy framework for multimodal closed-loop control. CTP models trajectory-level conditional

peaks over complete action chunks. One shared network forward pass provides multimodal prediction with $NFE = 1$, jointly addressing peak specialization and behavioral consistency across replanning cycles.

Our main contributions are as follows:

- We propose CTP, a single-pass framework for multimodal closed-loop control that jointly predicts multiple trajectory candidates, probabilities, and scales with $NFE = 1$.
- We introduce Distribution-Aware Peak Specialization (DAPS), which mitigates mode collapse through posterior responsibility assignment and probability- and scale-aware inter-peak regularization, without clustering or predefined mode assignments.
- We introduce Evidence-Gated Trajectory Belief Transport (ETBT), which improves cross-chunk behavioral consistency by propagating beliefs across replanning cycles while adaptively revising them using current policy evidence.

II. RELATED WORK

A. Multimodal Policies for Robotic Imitation Learning

Squared-error behavioral cloning tends toward the conditional mean and struggles with multivalued action mappings. Mixture density networks (MDNs) predict mixture parameters, providing a classical probabilistic framework for continuous multimodal regression [1]. With vectorized action chunks, a trajectory-level MDN uses one discrete component to explain the entire trajectory and optimizes a normalized mixture likelihood in one forward pass.

IBC represents implicit conditional distributions through an energy function and approximates inference by sampling actions [8]. BeT clusters continuous actions into discrete categories and predicts cluster indices and continuous residuals [9]. ACT shortens the effective prediction horizon through action chunking [10], while D3IL systematically evaluates how different policies learn diverse human demonstrations [11]. Energy Policy trains a noise-conditioned energy network to generate complete action sequences in parallel without iterative denoising [12]. Diffusion Policy and BESO demonstrate the effectiveness of iterative generation for multimodal robot control [2], [3]. These approaches improve distributional expressiveness, temporal modeling, or generation quality, but trade-offs remain among multimodal representation, policy inference overhead, and closed-loop candidate selection.

B. Single-Pass Multi-Hypothesis Generation

IMLE Policy regresses the demonstration’s closest candidate among action sequences generated from random latent variables, and uses the previous chunk’s unexecuted suffix for consistency-based selection [6]. PRISM combines full-batch rejection-sampling IMLE, Performer, and learned action queries to generate multiple sequences in one batched forward pass [7]. These studies establish single-pass multi-hypothesis generation’s feasibility. Liquid Networks with Mixture Density Heads predicts a Gaussian mixture per time

step [13]. Designs primarily trade off candidate sampling, learned action queries, and per-step mixtures. Direct parameterization of normalized complete action-chunk distributions in one shared forward pass, with one latent component spanning the entire horizon, remains insufficiently studied.

C. Consistency and Feedback across Action Chunks

BID samples multiple action chunks at inference, using backward coherence to preserve consistency with past decisions and forward contrast to guide candidate selection [14]. IntentVLA extracts short-horizon intent from recent visual history and conditions action-chunk generation on it, mitigating intent conflicts under observation aliasing [15]. IMLE Policy also uses the unexecuted suffix of the previous action chunk for consistency-based selection [6]. These studies highlight the importance of maintaining prior action intent. However, identifying the same physical route within candidate sets with permutable component indices, and actively releasing constraints when current evidence contradicts history, remain key challenges in receding-horizon execution.

D. Multimodal Trajectory Modeling in Motion Prediction

Motion prediction also represents multiple futures from limited history. AutoBots uses latent-variable set Transformers for single-pass scene-consistent trajectory prediction [16]; MotionDiffuser models joint multi-agent trajectory distributions through diffusion [17]. Multi-hypothesis regression commonly uses winner-takes-all objectives; aWTA stabilizes optimization through annealing [18], while ModeSeq explicitly models sequential relationships among modes [19]. Robotic closed-loop control shares trajectory-level multimodality with motion prediction but additionally handles receding-horizon replanning, contact feedback, and temporal permutations of finite mixture components. Single-step candidate diversity therefore cannot fully characterize closed-loop performance.

III. METHOD

A. Problem Formulation and Framework Overview

Given offline demonstrations $\mathcal{D} = \{(\mathcal{O}_t, A_t)\}$, let $\mathcal{O}_t = [o_{t-L+1}, \dots, o_t]$ denote the length- L observation history and $A_t = [a_t, \dots, a_{t+H-1}] \in \mathbb{R}^{H \times D}$ the target action chunk with horizon H and action dimension D . We seek a conditional policy $p_\theta(A_t | \mathcal{O}_t)$ representing distinct executable futures under the same observation.

Tasks may use absolute positions, relative displacements, or other action parameterizations. For trajectory modeling, we use the normalized representation $\hat{A}_t = \Psi_a(A_t; q_t)$, where q_t is the current action reference state and Ψ_a performs task-specific normalization and reference transformation. This unifies trajectory representations without predefining behavioral modes.

Our single-pass framework for multimodal closed-loop control, CTP, comprises three modules (Fig. 2): trajectory-level conditional peak modeling, DAPS, and ETBT. One shared backbone forward pass predicts complete trajectory candidates, probability masses, and scales. DAPS specializes

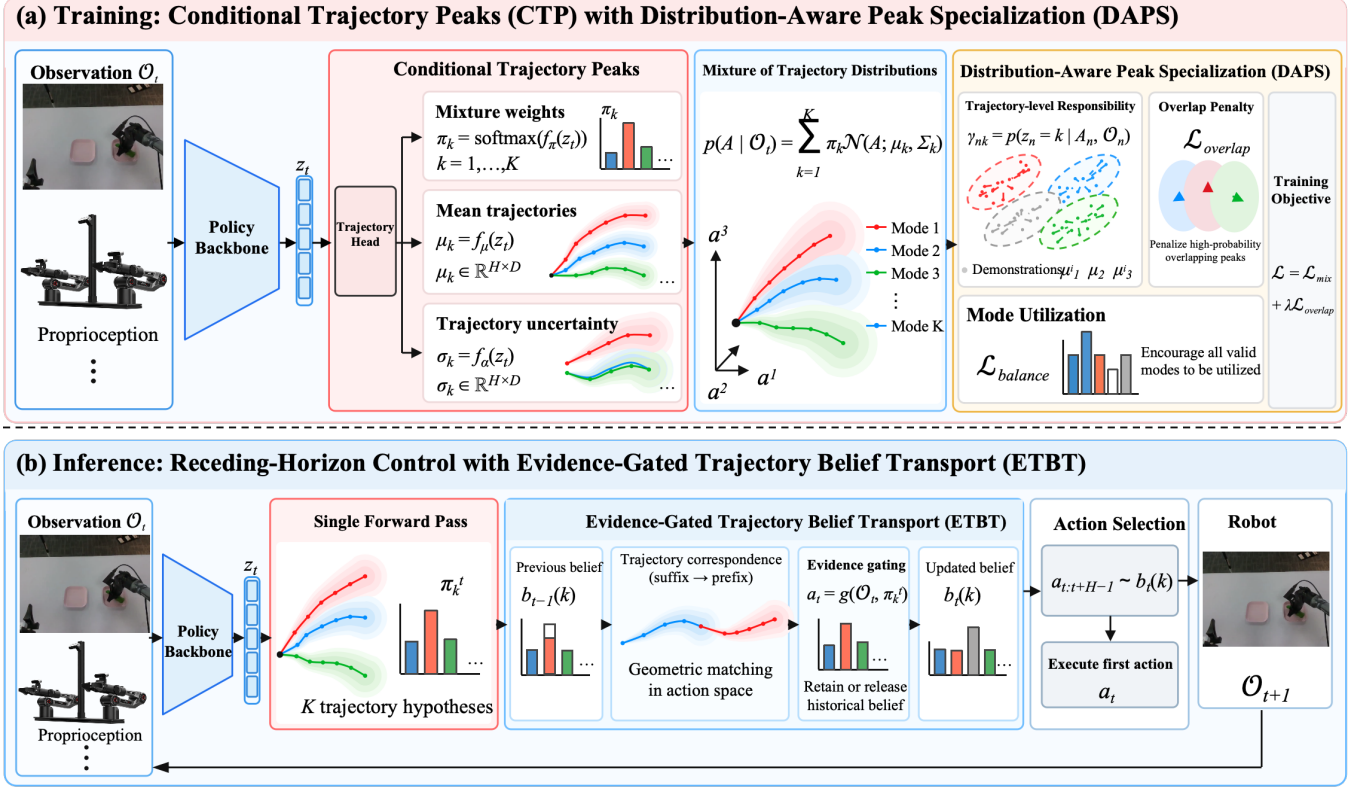


Fig. 2. Overview of CTP. (a) Training: the observation history passes through a shared policy backbone and trajectory head once to generate K trajectory peaks with probability masses and uncertainty. DAPS promotes mode specialization through trajectory-level responsibility assignment and inter-peak overlap constraints. (b) Inference: ETBT propagates historical behavioral beliefs through geometric correspondences between candidate trajectories in successive replanning cycles. Current policy evidence determines whether historical constraints are retained or released.

peaks during training; ETBT transfers behavioral beliefs across replanning cycles. Each module is detailed below.

B. Trajectory-Level Conditional Peak Modeling

The shared policy backbone first encodes the observation history into a context representation $z_t = f_\theta(N_o(\mathcal{O}_t)) \in \mathbb{R}^M$. The trajectory parameter head generates all K trajectory peaks in one shared forward pass. For peak k , it predicts trajectory parameters $C_{t,k} \in \mathbb{R}^{R \times D}$, a probability logit $\ell_{t,k}$, and a scale parameter $s_{t,k}$. Given a fixed trajectory basis $B \in \mathbb{R}^{H \times R}$, we define

$$\mu_{t,k} = BC_{t,k}, \quad \pi_t = \text{softmax}(\ell_t), \quad (1)$$

and

$$\sigma_{t,k} = \sigma_{\min} + (\sigma_{\max} - \sigma_{\min}) \text{sigmoid}(s_{t,k}). \quad (2)$$

Here, $\mu_{t,k}$ is the complete trajectory center of mode k , and $\pi_{t,k} = p(Z = k | \mathcal{O}_t)$ its probability mass under the current observation. The trajectory scale $\sigma_{t,k}$ describes conditional residual dispersion around that center in normalized action-chunk space. In particular, $\pi_{t,k}$ is not execution success probability, nor is $\sigma_{t,k}$ physical path width.

Using these parameters, CTP defines a finite conditional

mixture over complete action chunks:

$$p_\theta(\tilde{A} | \mathcal{O}) = \sum_{k=1}^K \pi_k(\mathcal{O}) \mathcal{N}(\text{vec}(\tilde{A}); \text{vec}(\mu_k), \sigma_k^2 I_{HD}). \quad (3)$$

The discrete variable $Z \in \{1, \dots, K\}$ selects one complete trajectory component to jointly explain the entire horizon. Mode identity therefore belongs to the complete action chunk, rather than varying independently at each step.

We call each component center μ_k a *trajectory peak*. The learned scale σ_k captures within-mode residual dispersion: larger residuals favor broader components, whereas the likelihood penalizes unrestricted inflation.

The basis B supports low-rank action-chunk parameterization ($R < H$, using a low-frequency basis) or direct time-domain prediction ($R = H$, $B = I_H$). Benchmark-specific choices appear in the experimental setup. Both retain a shared forward pass and $NFE = 1$ per replanning cycle.

C. DAPS

Finite mixtures provide capacity for multiple future behaviors, but more candidates alone do not ensure coverage of distinct data modes. Early in training, exchangeable candidates may converge to the demonstration distribution's dominant high-density region, producing nominally multiple but redundant peaks. DAPS addresses this mode collapse.

Let $d = HD$. For demonstration action chunk n and trajectory peak k , the Gaussian energy is

$$E_{nk} = \frac{\|\tilde{A}_n - \mu_{nk}\|_F^2}{2\sigma_{nk}^2} + d \log \sigma_{nk} + \frac{d}{2} \log(2\pi). \quad (4)$$

Marginalizing the discrete trajectory variable gives the mixture negative log-likelihood

$$\mathcal{L}_{\text{mix}} = -\frac{1}{N} \sum_{n=1}^N \log \sum_{k=1}^K \exp(\log \pi_{nk} - E_{nk}). \quad (5)$$

The corresponding trajectory-level posterior responsibility is $r_{nk} = \text{softmax}_k(\log \pi_{nk} - E_{nk})$.

Unlike hard winner-takes-all assignment based solely on trajectory distance, r_{nk} jointly considers fitting error, probability mass, and trajectory scale. It allocates learning signals consistently with the conditional mixture distribution without predefined candidate-to-mode mappings.

Soft responsibility assignment alone, however, does not ensure effective specialization. To further discourage high-probability peaks from repeatedly explaining the same data region, we define the normalized trajectory distance

$$d_{n,ij} = \frac{\|\mu_{ni} - \mu_{nj}\|_F^2}{HD}, \quad (6)$$

and construct an inter-peak overlap constraint

$$\mathcal{L}_{\text{ov}} = \frac{1}{N} \sum_{n=1}^N \sum_{i < j} \pi_{ni} \pi_{nj} \exp \left[-\frac{d_{n,ij}}{2(\sigma_{ni}^2 + \sigma_{nj}^2)} \right]. \quad (7)$$

This constraint induces distribution-aware selective specialization. The probability factor $\pi_{ni} \pi_{nj}$ concentrates separation pressure on pairs with high probability mass, while the scale term assesses overlap relative to their distributional widths. Unlike uniform repulsion in generic diversity regularization, DAPS suppresses data-supported, high-probability peaks that are mutually redundant. The overlap constraint encourages separation of independently supported candidates into distinct modes. Unsupported redundant candidates can reduce their probability mass and contribution, without forced assignment to artificially created additional modes.

The core DAPS training objective is

$$\mathcal{L}_{\text{DAPS}} = \mathcal{L}_{\text{mix}} + \lambda_{\text{ov}} \mathcal{L}_{\text{ov}}. \quad (8)$$

Trajectory-level posterior responsibilities assign candidate roles according to their ability to explain the data, while the distribution-aware overlap constraint discourages high-probability peaks from persistently explaining the same region. Together, they promote data-driven peak specialization. Backbone-specific auxiliary terms for candidate calibration or training stabilization appear in the experimental setup, outside the core DAPS definition.

D. Single-Pass Decoding and Receding-Horizon Execution

Given the current observation, CTP obtains all trajectory peaks in one forward pass. Without cross-cycle history, it uses maximum-probability decoding: $k_t^{\text{raw}} = \arg \max_k \pi_{t,k}$. The trajectory centers are mapped back to the task action

space as $\hat{A}_{t,k} = \Psi_a^{-1}(\mu_{t,k}; q_t)$. The policy executes the first $E \leq H$ actions of the selected chunk, then replans from new observations. Candidate generation requires no additional sampling, iterative denoising, or vector-field integration, so the procedure maintains $NFE = 1$.

E. ETBT

ETBT maintains behavioral identity during receding-horizon execution. Rather than merely smoothing actions, it explicitly matches behavior across trajectory sets with permutable indices and fuses historical behavior as a soft prior with current policy evidence.

First, a task-specific mapping Φ transforms candidate action chunk k into a trajectory representation that supports geometric comparison across cycles: $P_{t,k} = \Phi(\hat{A}_{t,k}; q_t)$. If E steps have been executed since the previous replanning cycle, let $M = H - E$. The correspondence cost between the unexecuted suffix of previous candidate i and the prefix of current candidate j is

$$C_t(i, j) = \frac{1}{MD_P} \sum_{h=1}^M \|P_{t-1,i,E+h} - P_{t,j,h}\|_2^2, \quad (9)$$

where D_P is the dimension of the representation used for trajectory matching.

We normalize the correspondence cost by a robust trajectory scale s_t and construct a similarity kernel $G_t(i, j) = \exp\left(-\frac{C_t(i, j)}{s_t \tau}\right)$, then obtain conditional correspondence probabilities using a normalization operator $\Gamma(\cdot)$ that preserves equivariance to candidate permutations:

$$T_t(j | i) = \Gamma(G_t)_{ij}, \quad \sum_j T_t(j | i) = 1. \quad (10)$$

Depending on the task, this can be implemented by row-wise conditional normalization or Sinkhorn normalization with marginal constraints.

Given the behavioral belief $b_{t-1}(i)$ over the previous candidate set, trajectory correspondences propagate a belief about behavioral continuation into the current cycle: $c_t(j) = \sum_i b_{t-1}(i) T_t(j | i)$. If only the previously executed candidate is retained, b_{t-1} reduces to a one-hot distribution on that candidate.

We smooth c_t into a historical behavioral prior $\bar{b}_t = \rho c_t + (1 - \rho)u$, where u is a weak fallback prior and ρ controls the strength of historical behavior retention. Current policy probabilities provide evidence under the latest observation: $q_t(j) = \pi_t(j)^\beta$. Fusing the historical prior and current policy evidence yields

$$b_t(j) = \frac{\bar{b}_t(j) q_t(j)}{\sum_m \bar{b}_t(m) q_t(m)}. \quad (11)$$

The key to ETBT is that history acts only as a soft prior, rather than an inviolable hard constraint. Let $j_c = \arg \max_j c_t(j)$, $j_q = \arg \max_j q_t(j)$. If the candidate most strongly supported by the current policy has substantially stronger evidence than the candidate continuing the historical behavior, namely, $\frac{q_t(j_q)}{q_t(j_c) + \epsilon} \geq \gamma$, or if the cross-cycle trajectory correspondence is unreliable, $\max_j c_t(j) <$

TABLE I

CLOSED-LOOP PERFORMANCE OF MULTIMODAL POLICIES ON PUSH-T.

Method	Reported results	Unified evaluation
IBC/DFO [2], [8]	0.90/0.84	0.8340
BeT [2], [9]	0.79/0.70	0.7817
Energy Policy [12]	0.85	0.7300
Diffusion Policy-C [2]	0.95/0.91	0.8790
IMLE Policy [6]	0.59/0.54	0.8874
Liquid-MDN [13]	0.91	0.7394
Trajectory-level MDN [1]	–	0.7963
CTP	–	0.9140

η , the historical constraint is released and candidate selection reverts to current policy evidence. Otherwise, the selected candidate is $k_t^* = \arg \max_j b_t(j)$.

ETBT thus maintains behavioral continuation across cycles without relying on fixed candidate indices. It releases historical constraints when current evidence is sufficiently strong or trajectory correspondences are unreliable, without additional policy backbone forward passes.

IV. EXPERIMENTS

A. Experimental Setup

Push-T: We use the Push-T closed-loop environment from Diffusion Policy [2]. Policies receive two historical states and predict 16×2 action chunks. CTP and the single-peak baseline use $K = 4/1$, respectively, with the same Transformer backbone, training data, and 40-epoch training configuration. After five training runs, the final EMA weights are evaluated on 200 fixed initial conditions.

D3IL: We use three state-based control tasks from D3IL: Avoiding, Aligning, and Sorting-2 [11]. Their multiple solutions arise from obstacle-avoidance paths, inner- versus outer-side alignment, and object sorting order, respectively. These tasks cover three representative multimodal control settings: path planning, pose alignment, and object sorting. They evaluate CTP’s ability to model different types of multimodal behavior and its policy selection.

LIBERO: We use the Spatial, Object, Goal, and Long suites of LIBERO [20], comprising 40 tasks. CTP freezes the $\pi_{0.5}$ vision-language backbone, uses $K = 4$, predicts 50-step action chunks, replans every 10 steps, and selects candidates through the belief decoder.

B. Main Results on Push-T

1) *Comparison with Existing Multimodal Policies:* Table I compares CTP with six multimodal policies: IBC/DFO, BeT, Energy Policy, Diffusion Policy-C, IMLE Policy, and Liquid-MDN, plus a standard trajectory-level MDN as a direct baseline. It includes published results and a unified evaluation. Because training and evaluation settings vary across studies, quantitative comparisons primarily use the unified results.

Under the unified protocol, CTP scores 0.9140, the highest closed-loop performance among compared methods. The strongest prior methods, IMLE Policy and Diffusion Policy-C, score 0.8874 and 0.8790, respectively. CTP improves over IMLE Policy by 0.0266 and trajectory-level MDN by

TABLE II

MODEL SIZE AND POLICY INVOCATION LATENCY ON THE SAME HARDWARE.

Method	Params. (M)	NFE	Mean (ms)	P95 (ms)
CTP	74.041	1	0.9507	0.9745
Diffusion Policy	65.783	100	902.8522	904.8401

TABLE III

CLOSED-LOOP SUCCESS RATES (%) ON D3IL MULTIMODAL CONTROL TASKS.

Task	BC-MLP	DDPM-MLP	Best prior result	CTP
Avoiding	66.6 $\pm$ 51.2	63.7 $\pm$ 5.5	BESO 95.0 $\pm$ 1.5	100.0
Aligning	70.8 $\pm$ 5.2	76.3 $\pm$ 3.9	VAE-ACT 89.1 $\pm$ 2.2	79.72
Sorting-2	44.4 $\pm$ 6.9	46.0 $\pm$ 3.9	DDPM-ACT 88.2 $\pm$ 2.3	84.44

0.1177, indicating that explicit trajectory-level candidates further improve multimodal closed-loop performance.

CTP predicts multiple trajectories with NFE= 1, without iterative sampling of multimodal actions. These results demonstrate effective single-pass trajectory-level multimodal modeling and strong Push-T closed-loop performance.

2) *Inference Efficiency:* Single-pass inference substantially reduces CTP’s online policy invocation cost. CTP with $K = 4$ has 74.041M parameters, more than the compared Diffusion Policy (65.783M); the latency difference therefore cannot be attributed to a smaller model. On the same RTX 5070, CTP’s mean and P95 invocation latencies are 0.9507 ms and 0.9745 ms, respectively. Diffusion Policy with NFE= 100 requires 902.8522 ms and 904.8401 ms, respectively. The efficiency difference primarily results from the number of network evaluations required for parallel single-pass output versus iterative generation.

C. Results on Key D3IL Tasks

Table III reports closed-loop success on three representative D3IL tasks. Avoiding, Aligning, and Sorting-2 admit multiple obstacle-avoidance paths, alignment strategies, and object sorting orders, respectively, testing different types of multimodal behavior modeling. Prior results primarily serve as performance references because training and evaluation protocols are not fully matched.

CTP achieves success rates of 100.0%, 79.72%, and 84.44% on Avoiding, Aligning, and Sorting-2, respectively. Its 100.0% success rate on Avoiding exceeds the best previously reported result of 95.0%. On Sorting-2, CTP achieves 84.44%, close to the best prior result of 88.2%.

In the example shown in Fig. 3, starting from the same initial state, the deterministic MLP collides with an obstacle, whereas CTP follows a collision-free route to the goal during closed-loop execution.

On Aligning, we test candidate count under strictly matched training and evaluation. Increasing $K = 1$ to $K = 4$ with all else fixed raises success from 69.17% to 79.72%: a 10.56 percentage-point gain, with a stratified 95% confidence interval of [0.28, 21.11] percentage points. Greater trajectory

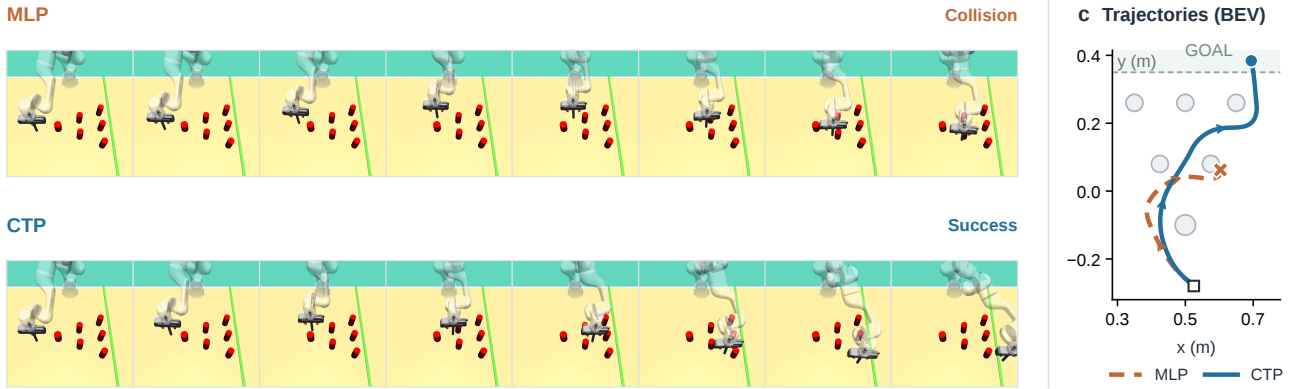


Fig. 3. Qualitative comparison on D3IL Avoiding. The deterministic MLP trained with MSE (top) and full CTP (bottom) start from the same initial state. Each row presents eight frames spanning a complete rollout; the bird’s-eye view (BEV) shows the corresponding executed trajectories. In this example, the MLP collides with an obstacle, while CTP successfully reaches the goal.

TABLE IV
COMPARISON OF RESULTS ON LIBERO.

Method	Spatial	Object	Goal	Long	Average
OpenVLA [21]	84.7	88.4	79.2	53.7	76.50
π_0 [22]	96.8	98.8	95.8	85.2	94.15
LingBot-VA [23]	98.5	99.6	97.2	98.5	98.45
Motus [24]	96.8	99.8	96.6	97.6	97.70
Fast-WAM [25]	98.2	100.0	97.0	95.2	97.60
$\pi_{0.5}$ [5]	97.0	99.0	98.0	93.0	96.75
CTP	98.0	100.0	98.0	93.0	97.25

candidate capacity thus improves multimodal behavior modeling and closed-loop execution on Aligning.

Overall, CTP achieves competitive closed-loop performance across tasks with distinct multimodal structures in path selection, alignment strategy, and execution order.

D. LIBERO: Transfer to Vision–Language–Action Models

To test CTP in robot manipulation conditioned on vision and language, we integrate it into $\pi_{0.5}$ and evaluate the same checkpoint on all four LIBERO suites. This experiment examines whether pretrained vision-language representations continue to support multitask closed-loop manipulation when parallel trajectory peak prediction replaces iterative action generation.

As shown in Table IV, CTP achieves success rates of 98.0%, 100.0%, 98.0%, and 93.0% on Spatial, Object, Goal, and Long, respectively, averaging 97.25%. This is close to Motus (97.70%) and Fast-WAM (97.60%), below LingBot-VA (98.45%), and above the locally reevaluated $\pi_{0.5}$ baseline (96.75%). Because these success rates do not come from strictly matched protocols, cross-model differences only contextualize overall performance. The experiment shows that structured single-pass action output can support LIBERO multitask closed-loop manipulation while retaining a large vision-language backbone.

CTP reduces mean latency for a complete policy invocation from 218.24 ms to 75.80 ms, a $2.88\times$ speedup and a 65.27% latency reduction. P95 latency decreases from

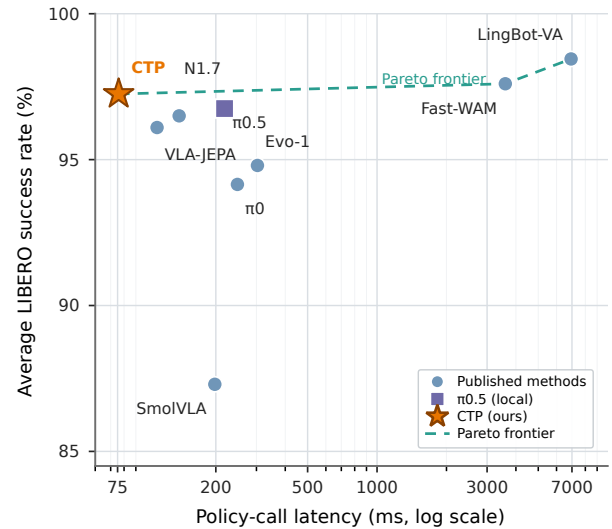


Fig. 4. Success–latency trade-off on LIBERO. CTP achieves an average success rate of 97.25% with a mean policy-call latency of 75.80 ms. All policy-call latencies were measured locally at batch size 1 on identical hardware comprising an NVIDIA GeForce RTX 4090 GPU and two Intel Xeon Gold 6530 CPUs. Success rates for published baselines are taken from their respective papers, whereas CTP and $\pi_{0.5}$ are evaluated under the same local protocol.

219.64 ms to 76.15 ms. The local $\pi_{0.5}$ baseline uses 10 flow integration steps per action chunk. CTP generates all trajectory peaks through one action expert forward pass, then selects a candidate using trajectory beliefs. Both predict 50 steps and execute 10, so the speedup does not result from a lower replanning frequency. Timing includes the vision-language backbone, data processing, and belief decoding. All policy-call latencies in Fig. 4 were measured by us on the same local hardware. Success rates for published baselines are taken from their original papers and are therefore not protocol-matched; CTP and the local $\pi_{0.5}$ baseline use our unified evaluation.

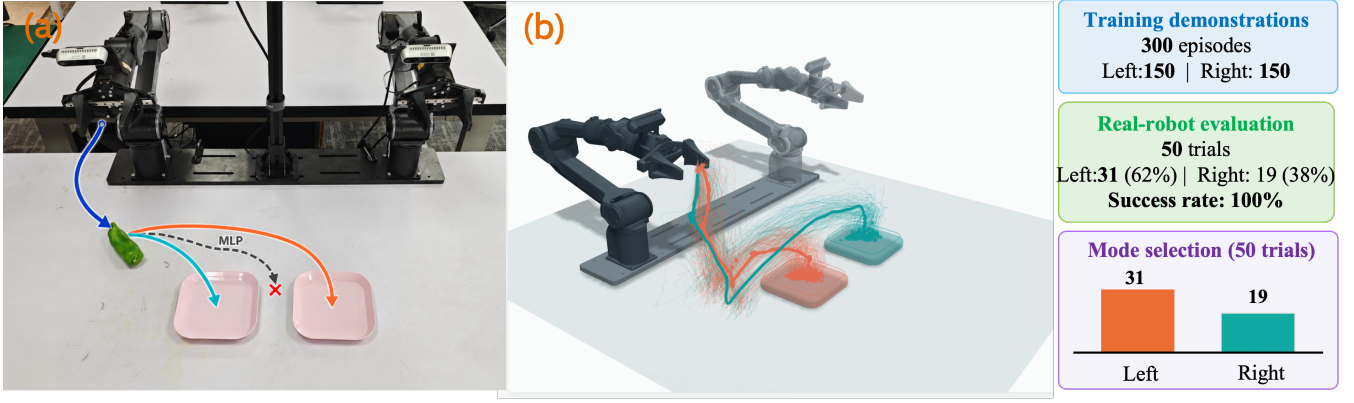


Fig. 5. Placement into either of two plates. (a) The real-world setup. After grasping the target object, the robot may place it in either the left or right plate; both outcomes count as success. (b) Demonstration trajectories visualized in a 3D scene model. Orange and cyan trajectories correspond to the left- and right-plate placement modes, respectively. We collected 300 demonstration episodes, with 150 per mode, yielding a balanced, spatially separated bimodal trajectory distribution.

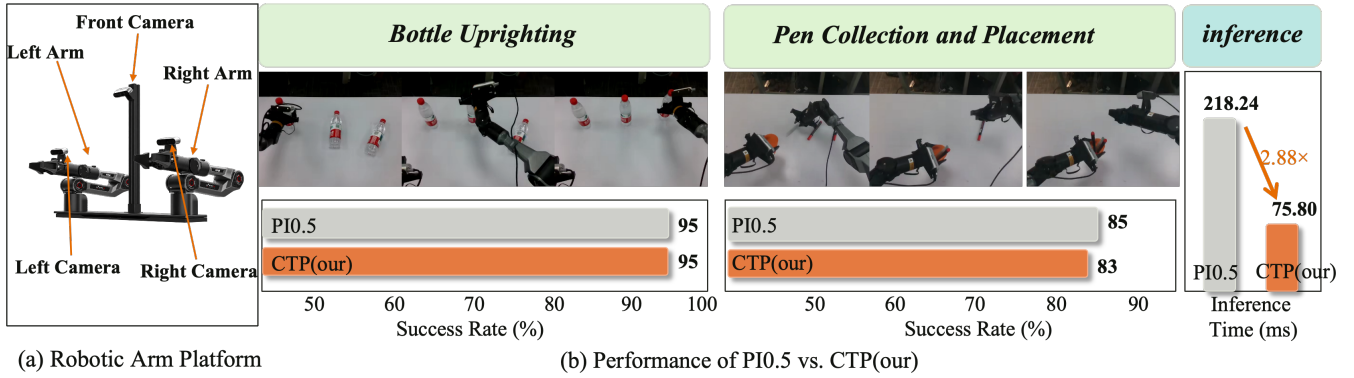


Fig. 6. Real-world bottle uprighting and pen placement experiments and comparative results. (a) Dual-arm robot platform and camera configuration. (b) Representative execution sequences, task success rates, and per-call policy inference latencies of $\pi_{0.5}$ and CTP on the same NVIDIA RTX 4090 GPU.

TABLE V
INCREMENTAL COMPONENT ABLATIONS OF CTP ON PUSH-T,
MEASURED BY MEAN SCORE (%).

Configuration	Mean Score (%)
Single-peak baseline ($K = 1$)	77.34
Multi-peak baseline ($K = 4$)	80.30
Multi-peak + DAPS	85.67
CTP (full)	91.40

E. Ablations of Key Components

To analyze each component’s contribution, we start from a single-peak baseline and successively introduce trajectory-level multimodal representation, DAPS, and ETBT (Table V). All configurations follow the training and evaluation protocol of the main experiments.

Increasing K from 1 to 4 raises Mean Score from 77.34% to 80.30%, a gain of 2.96 percentage points. This indicates that trajectory-level multimodal representation improves coverage of multimodal behavior. Adding DAPS further increases performance to 85.67%, a gain of 5.37 percentage points, indicating that peak specialization reduces candidate

redundancy and promotes differentiation among modes.

Finally, enabling ETBT for closed-loop candidate selection with the same model weights raises Mean Score to 91.40%, a further gain of 5.73 percentage points. This shows that, beyond obtaining candidate trajectories with good mode coverage, maintaining consistent behavioral choices during receding-horizon replanning is also key to improving closed-loop performance.

F. Real-World Robot Experiments

We evaluate real-world dual-arm closed-loop execution from two complementary perspectives: *multimodal behavior preservation* and *online inference efficiency*. All experiments share the robot and computing platform, using an NVIDIA RTX 4090 GPU for policy inference. Three tasks are considered: placement into either of two plates, bottle uprighting, and pen placement into a holder. The two-plate task directly tests preservation of spatially separated valid behavioral modes in real-world closed-loop execution; the other tasks compare success rates and online inference efficiency with $\pi_{0.5}$ [5].

1) *Placement into Either of Two Plates: Multimodal Behavior Preservation*: In Fig. 5(a), the robot grasps an object

and places it in either plate; both outcomes succeed. Instructions leave the plate unspecified, so identical observations and instructions admit two spatially separated executable modes. This directly tests CTP’s multimodal behavior preservation in real-world closed-loop execution.

We collected 300 demonstration episodes, with 150 for each placement mode. As shown in Fig. 5(b), the two sets of trajectories converge to the respective target regions, forming a balanced, spatially separated bimodal trajectory distribution. Across 50 real-world trials, CTP selects the left plate 31 times and the right plate 19 times (62% and 38%, respectively). All 50 placements succeed. No trial shows averaged trajectories converging between the targets. CTP thus preserves both valid modes in real-world closed-loop deployment, without collapsing to one mode or an intermediate compromise trajectory.

2) *Bottle Uprighting and Pen Placement: Task Success and Inference Efficiency:* We also compare CTP and $\pi_{0.5}$ on bottle uprighting and pen placement into a holder (Fig. 6). The former requires grasping fallen bottles at different positions and orientations and restoring them upright; the latter requires accurately placing tabletop pens into a holder after grasping. These assess object reorientation and precise placement, respectively. On bottle uprighting, both $\pi_{0.5}$ and CTP achieve a success rate of 95%. On pen placement, their success rates are 85% and 83%, respectively. All experiments use the same robot platform and NVIDIA RTX 4090 GPU. CTP reduces per-call policy inference latency from 218.24 ms to 75.80 ms, corresponding to a $2.88\times$ speedup. The results show that CTP substantially reduces online policy inference overhead while maintaining real-world success rates comparable to $\pi_{0.5}$.

V. CONCLUSION

We presented CTP, a single-pass policy framework for multimodal closed-loop control. CTP models trajectory-level multimodal distributions in the space of complete action chunks. It combines DAPS and ETBT to promote peak specialization and maintain cross-chunk behavioral consistency during receding-horizon replanning, respectively. The framework thus integrates multimodal representation, closed-loop continuity, and single-pass inference. Experiments on Push-T, D3IL, and LIBERO show competitive multimodal closed-loop performance and substantially reduced online inference overhead compared with iterative action generation. Real-world experiments further demonstrate that CTP preserves multiple valid behavioral modes during closed-loop execution and achieves a $2.88\times$ policy inference speedup while maintaining task success rates comparable to $\pi_{0.5}$. Ablations confirm the complementary contributions of trajectory-level multimodal representation, DAPS, and ETBT. Overall, CTP provides a trajectory-level modeling approach to multimodal imitation learning that combines behavioral diversity, closed-loop consistency, and online efficiency.

REFERENCES

- [1] C. M. Bishop, “Mixture density networks,” Aston University, Tech. Rep. NCRG/94/004, 1994.
- [2] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [3] M. Reuss, M. X. Li, X. Jia, and R. Lioutikov, “Goal-conditioned imitation learning using score-based diffusion policies,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.02747>
- [5] Physical Intelligence *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [6] K. Rana, R. Lee, D. Pershouse, and N. Sünderhauf, “IMLE policy: Fast and sample efficient visuomotor policy learning via implicit maximum likelihood estimation,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [7] A. Bhaskar, P. Tokekar, S. D. Cairano, and A. Schperberg, “PRISM: Performer RS-IMLE for single-pass multisensory imitation learning,” *arXiv preprint arXiv:2602.02396*, 2026.
- [8] P. Florence *et al.*, “Implicit behavioral cloning,” in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164, 2022, pp. 158–168.
- [9] N. M. Shafiuallah, Z. Cui, A. Altanzaya, and L. Pinto, “Behavior transformers: Cloning k modes with one stone,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [10] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [11] X. Jia *et al.*, “Towards diverse behaviors: A benchmark for imitation learning with human demonstrations,” in *International Conference on Learning Representations*, 2024.
- [12] J. Jia, T. Yang, X. Chen, C. Liu, and W. Zhang, “Fast visuomotor policy for robotic manipulation,” *arXiv preprint arXiv:2510.12483*, 2025.
- [13] N. Correll, “Liquid networks with mixture density heads for efficient imitation learning,” *arXiv preprint arXiv:2603.27058*, 2026.
- [14] Y. Liu, J. I. Hamid, A. Xie, Y. Lee, M. Du, and C. Finn, “Bidirectional decoding: Improving action chunking via guided test-time sampling,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://bid-robot.github.io/>
- [15] S. Lian *et al.*, “IntentVLA: Short-horizon intent modeling for aliased robot manipulation,” *arXiv preprint arXiv:2605.14712*, 2026.
- [16] R. Girgis *et al.*, “Latent variable sequential set transformers for joint multi-agent motion prediction,” in *International Conference on Learning Representations*, 2022.
- [17] C. M. Jiang, A. Cornman, C. Park, B. Sapp, Y. Zhou, and D. Anguelov, “Motiondiffuser: Controllable multi-agent motion prediction using diffusion,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 9644–9653.
- [18] Y. Xu, V. Letzelter, M. Chen, E. Zabolocki, and M. Cord, “Annealed winner-takes-all for motion forecasting,” in *IEEE International Conference on Robotics and Automation*, 2025.
- [19] Z. Zhou *et al.*, “Modeseq: Taming sparse multimodal motion prediction with sequential mode modeling,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 1612–1621.
- [20] B. Liu *et al.*, “LIBERO: Benchmarking knowledge transfer for life-long robot learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [21] M. J. Kim *et al.*, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.09246>
- [22] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.24164>
- [23] L. Li *et al.*, “Causal world modeling for robot control,” *arXiv preprint arXiv:2601.21998*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.21998>
- [24] H. Bi *et al.*, “Motus: A unified latent action world model,” *arXiv preprint arXiv:2512.13030*, 2025. [Online]. Available: <https://arxiv.org/abs/2512.13030>
- [25] T. Yuan, Z. Dong, Y. Liu, and H. Zhao, “Fast-WAM: Do world action models need test-time future imagination?” *arXiv preprint arXiv:2603.16666*, 2026, version 1. [Online]. Available: <https://arxiv.org/abs/2603.16666v1>